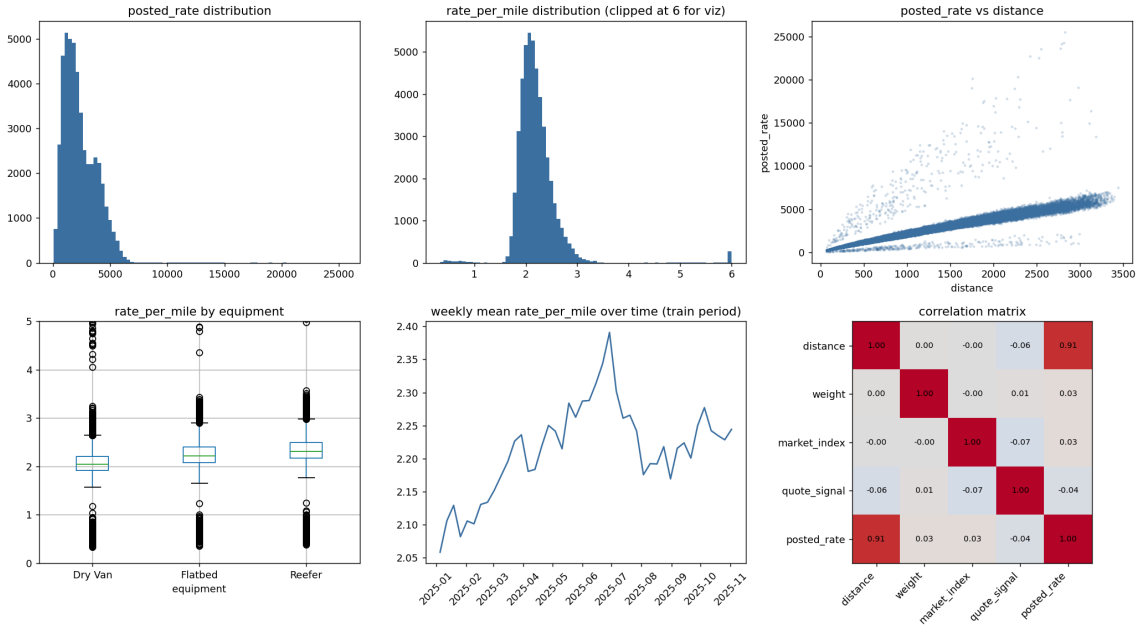
Part 1 — Key Findings from Exploring the Data

- **48,000** labeled loads in train-test.csv (Jan–Oct 2025), **12,000** unlabeled loads to score in validation.csv (Nov–Dec 2025). No duplicate rows or load_ids in either file.
- **distance** is the dominant predictor of posted_rate (r = 0.91), and is ~99.95% correlated with the great-circle (haversine) distance computed from the lat/lon columns — confirming it is a trustworthy, uncorrupted feature.
- **equipment** shifts price: mean rate-per-mile is Dry Van $2.12 < Flatbed $2.29 < Reefer $2.38.
- **market_index** and **quote_signal** have weak linear correlation with posted_rate (|r| < 0.07) individually, but retain some independent signal confirmed later via permutation importance.
- A mild **seasonal trend**: mean rate-per-mile drifts upward from ~2.05 in January to a peak around ~2.39 mid-year, then eases back down.
- **8 cities in validation.csv never appear in train-test.csv** (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) — a critical generalization constraint that shaped the feature engineering (see Part 2).
- The target is **right-skewed** with a small tail (~0.5%) of extreme outlier loads at rate-per-mile up to ~14, versus a median of ~2.1.



EDA overview: posted_rate and rate-per-mile distributions, posted_rate vs. distance, rate-per-mile by equipment, weekly seasonal trend, and the numeric correlation matrix.

Part 2 — Data-Quality Issues Identified & How They Were Addressed

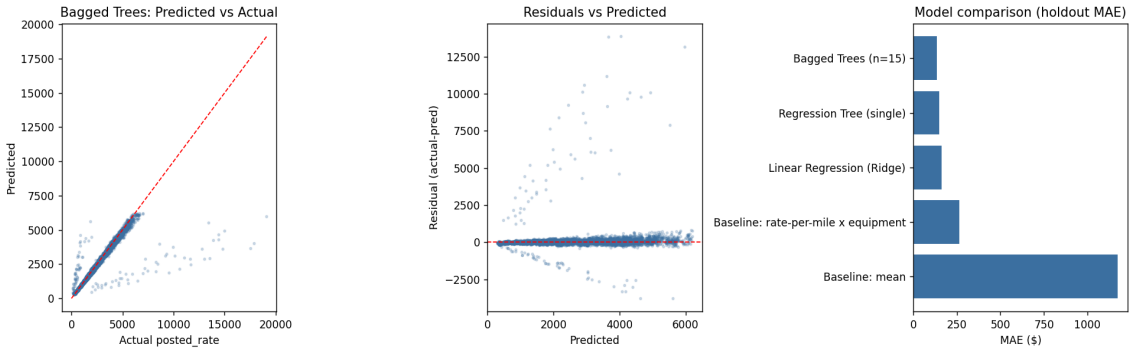
Issue	Scope	Fix	Reasoning
Negative weight values	~0.6% of rows (train & val)	Took abs(weight)	abs(negative) distribution almost exactly matches the valid positive
Missing weight	~0.6% of rows	Imputed with training-fold median	Missingness is roughly proportional across equipment types — cor
Missing market_index	~0.8% of rows	Imputed with training-fold median + missing-flag feature	Same MCAR-like pattern as weight; the flag lets the model use “wa
8 unseen cities in validation	8 of ~70 cities	Dropped city-name categoricals; used location-level encodings only	Out-of-fold target-encoded city features have no valid encoding for
Extreme target outliers	~0.5% of rows	Winsorized training target at [0.5%, 99.5%] quantiles; capped leverage of extreme-outlier loads on the fitte

Part 3 — Reasoning Behind the Chosen Model

requirement.txt lists only pandas, numpy, and matplotlib (no scikit-learn), so every model below — including the baselines — is implemented from scratch in numpy (see **features_models.py** in Part 6).

Model	MAE ($)	RMSE ($)	R2	MAPE (%)
Baseline: mean	1,173.9	1,515.7	-0.00	82.9
Baseline: rate-per-mile × equipment	263.3	666.4	0.807	11.1
Linear Regression (Ridge)	162.7	616.9	0.834	8.7
Regression Tree (single)	149.0	615.3	0.835	6.9
Bagged Trees (n=25) — selected	**135.0**	**610.0**	**0.838**	**6.6**

Bagged Trees selected. Best MAE / RMSE / R^2 / MAPE on the out-of-time holdout (see Part 4). It captures non-linear equipment × distance × region interactions the linear model can't, while a 25-tree ensemble of shallow trees stays simple enough to remain interpretable via permutation importance — avoiding the jump to a much heavier model for a modest additional gain.



Left/middle: predicted-vs-actual and residuals for the selected bagged-trees model on the holdout. Right: holdout MAE by model.

Permutation importance (bagged trees, increase in holdout MAE when a feature is shuffled):

Feature	MAE increase ($)
distance	1,354.7
eq_Reefer	46.3
pickup_lon	28.9
delivery_lon	21.4
quote_signal	20.6
eq_Flatbed	10.0
weight_clean	4.8

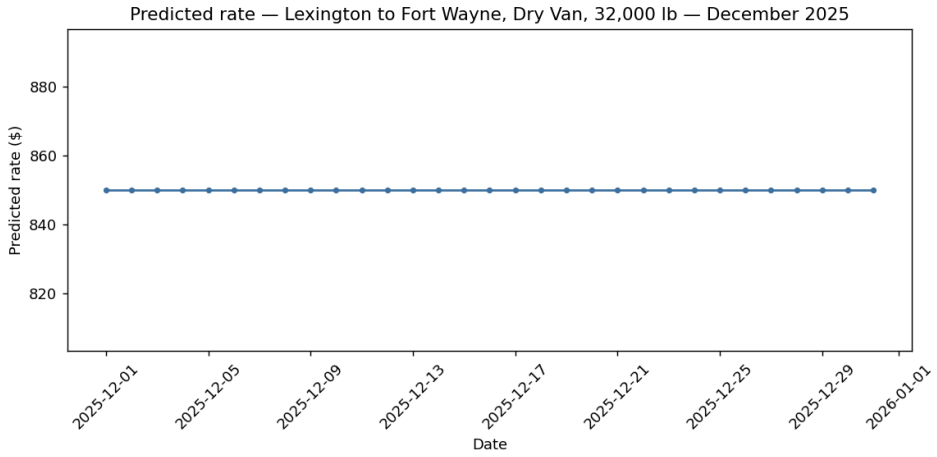
pickup_lat / delivery_lat / month / market_index / missing-flags	< 1 each

Part 4 — Training and Validation Approach

validation.csv is entirely out-of-time relative to train-test.csv (Nov–Dec 2025 vs. Jan–Oct 2025). A random split would let the model see rows from every month during training and overstate how well it generalizes to genuinely future data. A time-based holdout was used instead:

Fold	Date range	Row count	Role
Development-train	2025-01-01 → 2025-09-15	40,850	Fit imputers, winsorize target, train model
Development-holdout	2025-09-16 → 2025-10-31	7,150	Measure honest out-of-time accuracy

- **No leakage across the split:** imputation medians and the target winsorization bounds are computed only on the training fold, then applied unchanged to the holdout fold and, later, to validation.csv.
- **Split validated before trusting it:** row counts/date boundaries were confirmed with no overlap; a trivial mean-only baseline was scored first (MAE $1,174, $R^2 \approx 0$) to confirm the evaluation pipeline wasn't leaking before any real model was judged against it.
- **Final refit:** once bagged trees was selected on the holdout, it was refit from scratch on 100% of train-test.csv (48,000 rows) — not reused from the holdout run — before scoring validation.csv, maximizing the data available to the production model while keeping model *selection* strictly honest.



Bonus deliverable: predicted rate for a fixed lane across December 2025, generated by the same final model. It is flat by design — every feature the model uses is identical across all 31 rows (see REPORT.md and the separate PDF/DOCX for the full explanation).

Part 5 — Code Walkthrough (Summary)

File	Purpose
`features_models.py`	Feature engineering (imputation, cyclical month encoding, lat/lon-based location features) plus from-scrat
`01_eda_prep.py`	Loads all four input files, prints shapes/dtypes/missingness/duplicates/cardinality, checks the distance col
`02_train_eval.py`	Builds the time-based holdout split, fits imputers on the training fold only, trains and compares all baseline
`03_final_predict.py`	Refits the selected model on 100% of train-test.csv, scores validation.csv into validation_predictions.csv,

Full source for all four files follows in Part 6, in run order.

Part 6 — Complete Code Listings

features_models.py

```
"""
Reusable feature engineering + from-scratch models (numpy/pandas only,
per requirement.txt which does not include scikit-learn).
"""
import numpy as np
import pandas as pd

RANDOM_SEED = 42

FEATURE_COLS = [
    'distance', 'pickup_lat', 'pickup_lon', 'delivery_lat', 'delivery_lon',
    'weight_clean', 'market_index_clean', 'quote_signal',
    'eq_Reefer', 'eq_Flatbed', 'month_sin', 'month_cos',
    'weight_missing', 'market_missing'
]

def fit_imputers(df):
    """Compute imputation statistics from TRAINING data only (no leakage)."""
    weight_abs = df['weight'].abs()
    stats = {
        'weight_median': weight_abs.median(),
        'market_median': df['market_index'].median(),
    }
    return stats

def engineer_features(df, stats):
    df = df.copy()
    df['weight_missing'] = df['weight'].isnull().astype(float)
    df['weight_clean'] = df['weight'].abs()
    df['weight_clean'] = df['weight_clean'].fillna(stats['weight_median'])

    df['market_missing'] = df['market_index'].isnull().astype(float)
    df['market_index_clean'] = df['market_index'].fillna(stats['market_median'])

    df['eq_Reefer'] = (df['equipment'] == 'Reefer').astype(float)
    df['eq_Flatbed'] = (df['equipment'] == 'Flatbed').astype(float)

    month = df['date'].dt.month.astype(float)
    df['month_sin'] = np.sin(2*np.pi*month/12)
    df['month_cos'] = np.cos(2*np.pi*month/12)

    return df

def to_matrix(df):
    X = df[FEATURE_COLS].to_numpy(dtype=float)
    return X


# ---------------------------------------------------------------------
# Model 1: Ridge Linear Regression (closed-form, numpy only)
# ---------------------------------------------------------------------
class RidgeRegressionNumpy:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.mean_ = None
        self.std_ = None
        self.coef_ = None
        self.intercept_ = None

    def fit(self, X, y):
        self.mean_ = X.mean(axis=0)
        self.std_ = X.std(axis=0)
        self.std_[self.std_ == 0] = 1.0
        Xs = (X - self.mean_) / self.std_
        Xb = np.hstack([np.ones((Xs.shape[0], 1)), Xs])
        n_features = Xb.shape[1]
        reg = self.alpha * np.eye(n_features)
        reg[0, 0] = 0.0  # do not regularize intercept
```

```
        theta = np.linalg.solve(Xb.T @ Xb + reg, Xb.T @ y)
        self.intercept_ = theta[0]
        self.coef_ = theta[1:]
        return self

    def predict(self, X):
        Xs = (X - self.mean_) / self.std_
        return self.intercept_ + Xs @ self.coef_


# ---------------------------------------------------------------------
# Model 2: CART Regression Tree (from scratch, numpy only)
# ---------------------------------------------------------------------
class TreeNode:
    __slots__ = ['feature', 'threshold', 'left', 'right', 'value']
    def __init__(self, value=None):
        self.feature = None
        self.threshold = None
        self.left = None
        self.right = None
        self.value = value

class RegressionTreeNumpy:
    def __init__(self, max_depth=6, min_samples_leaf=100, min_samples_split=200,
                 max_features=None, random_state=None):
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.rng = np.random.RandomState(random_state)
        self.root = None

    def _best_split(self, X, y, feat_idx):
        n = len(y)
        best_gain, best_feat, best_thr = -np.inf, None, None
        parent_sse = np.sum((y - y.mean())**2)

        for f in feat_idx:
            col = X[:, f]
            order = np.argsort(col)
            col_sorted = col[order]
            y_sorted = y[order]

            # candidate split points: skip duplicate values
            distinct = np.where(np.diff(col_sorted) > 1e-12)[0]
            if len(distinct) == 0:
                continue

            cs = np.cumsum(y_sorted)
            cs2 = np.cumsum(y_sorted**2)
            total_sum, total_sq = cs[-1], cs2[-1]

            idxs = distinct  # index i means split after position i (0-indexed)
            left_n = idxs + 1
            right_n = n - left_n
            valid = (left_n >= self.min_samples_leaf) & (right_n >= self.min_samples_leaf)
            if not np.any(valid):
                continue
            idxs = idxs[valid]; left_n = left_n[valid]; right_n = right_n[valid]

            left_sum = cs[idxs]; left_sq = cs2[idxs]
            right_sum = total_sum - left_sum; right_sq = total_sq - left_sq

            left_sse = left_sq - (left_sum**2)/left_n
            right_sse = right_sq - (right_sum**2)/right_n
            gain = parent_sse - (left_sse + right_sse)

            best_local = np.argmax(gain)
            if gain[best_local] > best_gain:
                best_gain = gain[best_local]
                best_feat = f
                pos = idxs[best_local]
                best_thr = (col_sorted[pos] + col_sorted[pos+1]) / 2.0
```

```
        return best_feat, best_thr, best_gain

    def _build(self, X, y, depth):
        node = TreeNode(value=y.mean())
        if depth >= self.max_depth or len(y) < self.min_samples_split:
            return node

        n_feats = X.shape[1]
        if self.max_features is not None:
            feat_idx = self.rng.choice(n_feats, size=min(self.max_features, n_feats), replace=False)
        else:
            feat_idx = np.arange(n_feats)

        feat, thr, gain = self._best_split(X, y, feat_idx)
        if feat is None or gain <= 1e-9:
            return node

        mask = X[:, feat] <= thr
        node.feature = feat
        node.threshold = thr
        node.left = self._build(X[mask], y[mask], depth+1)
        node.right = self._build(X[~mask], y[~mask], depth+1)
        return node

    def fit(self, X, y):
        self.root = self._build(X, y, 0)
        return self

    def _predict_row(self, row, node):
        while node.feature is not None:
            node = node.left if row[node.feature] <= node.threshold else node.right
        return node.value

    def predict(self, X):
        return np.array([self._predict_row(row, self.root) for row in X])


class BaggedTreesNumpy:
    """Small random-forest-style ensemble of RegressionTreeNumpy."""
    def __init__(self, n_estimators=15, max_depth=7, min_samples_leaf=80,
                 min_samples_split=160, max_features=8, random_state=42):
        self.n_estimators = n_estimators
        self.trees = []
        self.params = dict(max_depth=max_depth, min_samples_leaf=min_samples_leaf,
                           min_samples_split=min_samples_split, max_features=max_features)
        self.random_state = random_state

    def fit(self, X, y):
        rng = np.random.RandomState(self.random_state)
        n = X.shape[0]
        self.trees = []
        for i in range(self.n_estimators):
            boot_idx = rng.randint(0, n, size=n)
            tree = RegressionTreeNumpy(random_state=self.random_state + i, **self.params)
            tree.fit(X[boot_idx], y[boot_idx])
            self.trees.append(tree)
        return self

    def predict(self, X):
        preds = np.column_stack([t.predict(X) for t in self.trees])
        return preds.mean(axis=1)


# ------------------------------------------------------------------
# Metrics
# ------------------------------------------------------------------
def regression_metrics(y_true, y_pred):
    err = y_true - y_pred
    mae = np.mean(np.abs(err))
    rmse = np.sqrt(np.mean(err**2))
    ss_res = np.sum(err**2)
    ss_tot = np.sum((y_true - y_true.mean())**2)
    r2 = 1 - ss_res/ss_tot
```

```
mape = np.mean(np.abs(err) / y_true) * 100
return {'MAE': mae, 'RMSE': rmse, 'R2': r2, 'MAPE': mape}
```

01_eda_prep.py

```
"""
Freight Load Rate Prediction — EDA & Preprocessing
Only uses numpy / pandas / matplotlib, per requirement.txt
"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

train = pd.read_csv("train-test.csv", parse_dates=["date"])
val   = pd.read_csv("validation.csv", parse_dates=["date"])
tmpl  = pd.read_csv("validation-predictions-template.csv")
chart = pd.read_csv("december-chart-inputs.csv", parse_dates=["date"])

print("="*70)
print("SHAPES")
print("="*70)
print("train-test.csv :", train.shape)
print("validation.csv :", val.shape)
print("template       :", tmpl.shape)
print("chart inputs   :", chart.shape)

print("\n" + "="*70)
print("TRAIN DTYPES")
print("="*70)
print(train.dtypes)

print("\n" + "="*70)
print("MISSING VALUES (train / val)")
print("="*70)
print(pd.DataFrame({"train": train.isnull().sum(), "val":
    val.reindex(columns=train.columns.drop('posted_rate')).isnull().sum() if False else
    val.isnull().sum().reindex(train.columns.drop('posted_rate'))}))

print("\nDuplicate rows (train):", train.duplicated().sum())
print("Duplicate load_id (train):", train['load_id'].duplicated().sum())
print("Duplicate load_id (val):", val['load_id'].duplicated().sum())

print("\n" + "="*70)
print("CARDINALITY")
print("="*70)
for c in ["pickup","delivery","equipment"]:
    print(c, "-> train unique:", train[c].nunique(), "| val unique:", val[c].nunique())

new_pickup = set(val['pickup']) - set(train['pickup'])
new_deliv  = set(val['delivery']) - set(train['delivery'])
print("\nCities present in validation but NEVER seen in train (pickup):", sorted(new_pickup))
print("Cities present in validation but NEVER seen in train (delivery):", sorted(new_deliv))

print("\n" + "="*70)
print("DATE RANGES")
print("="*70)
print("train:", train['date'].min().date(), "->", train['date'].max().date())
print("val  :", val['date'].min().date(),   "->", val['date'].max().date())
print("chart:", chart['date'].min().date(), "->", chart['date'].max().date())

print("\n" + "="*70)
print("WEIGHT SANITY CHECK (negative values)")
print("="*70)
print("train negative weight rows:", (train['weight']<0).sum(), "/",
    train['weight'].notnull().sum())
print("val   negative weight rows:", (val['weight']<0).sum(), "/", val['weight'].notnull().sum())
print("abs(negative) describe (train):\n", train.loc[train['weight']<0,'weight'].abs().describe())
print("positive describe (train):\n", train.loc[train['weight']>=0,'weight'].describe())

print("\n" + "="*70)
print("TARGET DISTRIBUTION: posted_rate")
print("="*70)
print(train['posted_rate'].describe())
```

```
train['rate_per_mile'] = train['posted_rate'] / train['distance']
print("\nrate_per_mile describe:\n", train['rate_per_mile'].describe())
print("\nrate_per_mile quantiles (tails):")
print(train['rate_per_mile'].quantile([0.0005,0.001,0.005,0.01,0.99,0.995,0.999,0.9995]))

print("\n" + "="*70)
print("EQUIPMENT EFFECT ON rate_per_mile")
print("="*70)
print(train.groupby('equipment')['rate_per_mile'].agg(['count','mean','median','std']))

print("\n" + "="*70)
print("CORRELATIONS WITH posted_rate")
print("="*70)
num_cols = ['distance','weight','market_index','quote_signal','posted_rate']
print(train[num_cols].corr()['posted_rate'])

# haversine sanity check for distance column
def haversine(lat1, lon1, lat2, lon2):
    R = 3958.8
    p1, p2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2-lat1); dlambda = np.radians(lon2-lon1)
    a = np.sin(dphi/2)**2 + np.cos(p1)*np.cos(p2)*np.sin(dlambda/2)**2
    return 2*R*np.arcsin(np.sqrt(a))
train['hav_dist'] = haversine(train.pickup_lat, train.pickup_lon, train.delivery_lat,
    train.delivery_lon)
print("\ncorr(distance, haversine great-circle distance):",
    train['distance'].corr(train['hav_dist']))

# ---------------- PLOTS ----------------
fig, axes = plt.subplots(2, 3, figsize=(16, 9))

axes[0,0].hist(train['posted_rate'], bins=80, color='#3B6FA0')
axes[0,0].set_title('posted_rate distribution')

axes[0,1].hist(train['rate_per_mile'].clip(upper=6), bins=80, color='#3B6FA0')
axes[0,1].set_title('rate_per_mile distribution (clipped at 6 for viz)')

axes[0,2].scatter(train['distance'], train['posted_rate'], s=3, alpha=0.15, color='#3B6FA0')
axes[0,2].set_xlabel('distance'); axes[0,2].set_ylabel('posted_rate')
axes[0,2].set_title('posted_rate vs distance')

train.boxplot(column='rate_per_mile', by='equipment', ax=axes[1,0])
axes[1,0].set_ylim(0, 5)
axes[1,0].set_title('rate_per_mile by equipment'); plt.suptitle('')

monthly = train.set_index('date')['rate_per_mile'].resample('W').mean()
axes[1,1].plot(monthly.index, monthly.values, color='#3B6FA0')
axes[1,1].set_title('weekly mean rate_per_mile over time (train period)')
axes[1,1].tick_params(axis='x', rotation=45)

corr = train[num_cols].corr()
im = axes[1,2].imshow(corr, vmin=-1, vmax=1, cmap='coolwarm')
axes[1,2].set_xticks(range(len(num_cols))); axes[1,2].set_xticklabels(num_cols, rotation=45,
    ha='right')
axes[1,2].set_yticks(range(len(num_cols))); axes[1,2].set_yticklabels(num_cols)
for i in range(len(num_cols)):
    for j in range(len(num_cols)):
        axes[1,2].text(j, i, f"{corr.values[i,j]:.2f}", ha='center', va='center', fontsize=8)
axes[1,2].set_title('correlation matrix')

plt.tight_layout()
plt.savefig('eda_overview.png', dpi=130)
print("\nSaved eda_overview.png")
```

02_train_eval.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from features_models import (fit_imputers, engineer_features, to_matrix,
                             RidgeRegressionNumpy, RegressionTreeNumpy, BaggedTreesNumpy,
                             regression_metrics, FEATURE_COLS)

np.random.seed(42)

train = pd.read_csv("train-test.csv", parse_dates=["date"])

# ---------------------------------------------------------------------
# TIME-BASED SPLIT
# Deployment validation.csv is strictly FUTURE (Nov-Dec) relative to
# train-test.csv (Jan-Oct). We mimic this with an internal time-based
# holdout: train on Jan 1 - Sep 15, validate on Sep 16 - Oct 31.
# A random split would overestimate performance because it would not
# test the model's ability to extrapolate forward in time / to new
# lanes, which is exactly the situation validation.csv presents.
# ---------------------------------------------------------------------
cutoff = pd.Timestamp('2025-09-16')
dev_train = train[train['date'] < cutoff].reset_index(drop=True)
dev_val   = train[train['date'] >= cutoff].reset_index(drop=True)
print(f"dev_train rows: {len(dev_train)}  ({dev_train['date'].min().date()} ->
    {dev_train['date'].max().date()})")
print(f"dev_val   rows: {len(dev_val)}  ({dev_val['date'].min().date()} ->
    {dev_val['date'].max().date()})")

# Fit imputation stats on TRAIN ONLY (avoid leakage)
stats = fit_imputers(dev_train)
print("Imputation stats (fit on training fold only):", stats)

dev_train_fe = engineer_features(dev_train, stats)
dev_val_fe   = engineer_features(dev_val, stats)

# Winsorize target for TRAINING rows only (cap extreme outliers that look
# like data-entry errors / rare extreme loads) - never touch val/test targets,
# we always evaluate against the real observed value.
lo, hi = dev_train_fe['posted_rate'].quantile([0.005, 0.995])
y_train_raw = dev_train_fe['posted_rate'].to_numpy()
y_train_w = np.clip(y_train_raw, lo, hi)
print(f"Winsorizing training target at [{lo:.1f}, {hi:.1f}] "
      f"affects {(np.abs(y_train_raw-y_train_w)>1e-6).sum()} / {len(y_train_raw)} training rows")

X_train = to_matrix(dev_train_fe)
X_val   = to_matrix(dev_val_fe)
y_val   = dev_val_fe['posted_rate'].to_numpy()

results = {}

# ---------------- Baseline 1: global mean ----------------
pred_mean = np.full_like(y_val, y_train_w.mean())
results['Baseline: mean'] = regression_metrics(y_val, pred_mean)

# ---------------- Baseline 2: mean rate-per-mile x distance x equipment ----------------
rpm = (dev_train_fe['posted_rate'] / dev_train_fe['distance'])
eq_rpm = rpm.groupby(dev_train_fe['equipment']).mean()
overall_rpm = rpm.mean()
pred_rpm = dev_val_fe['equipment'].map(eq_rpm).fillna(overall_rpm).to_numpy() *
    dev_val_fe['distance'].to_numpy()
results['Baseline: rate-per-mile x equipment'] = regression_metrics(y_val, pred_rpm)

# ---------------- Model A: Ridge Linear Regression ----------------
lin = RidgeRegressionNumpy(alpha=5.0)
lin.fit(X_train, y_train_w)
pred_lin = lin.predict(X_val)
results['Linear Regression (Ridge)'] = regression_metrics(y_val, pred_lin)

# ---------------- Model B: single Regression Tree ----------------
tree = RegressionTreeNumpy(max_depth=8, min_samples_leaf=60, min_samples_split=120)
tree.fit(X_train, y_train_w)
```

```
pred_tree = tree.predict(X_val)
results['Regression Tree (single)'] = regression_metrics(y_val, pred_tree)

# ---------------- Model C: Bagged Trees (small random forest) ----------------
bag = BaggedTreesNumpy(n_estimators=15, max_depth=8, min_samples_leaf=60,
                       min_samples_split=120, max_features=10, random_state=42)
bag.fit(X_train, y_train_w)
pred_bag = bag.predict(X_val)
results['Bagged Trees (n=15)'] = regression_metrics(y_val, pred_bag)

print("\n" + "="*70)
print("INTERNAL TIME-BASED HOLDOUT RESULTS")
print("="*70)
res_df = pd.DataFrame(results).T[['MAE','RMSE','R2','MAPE']]
print(res_df.round(3))
res_df.to_csv('model_comparison.csv')

# Linear regression coefficients (standardized) for interpretability
coef_df = pd.DataFrame({'feature': FEATURE_COLS, 'std_coef': lin.coef_}).sort_values('std_coef',
    key=abs, ascending=False)
print("\nRidge regression standardized coefficients:\n", coef_df)
coef_df.to_csv('linear_coefficients.csv', index=False)

# ---------------- Permutation importance for the bagged-trees model ----------------
baseline_mae = regression_metrics(y_val, pred_bag)['MAE']
importances = []
rng = np.random.RandomState(0)
for i, feat in enumerate(FEATURE_COLS):
    Xp = X_val.copy()
    rng.shuffle(Xp[:, i])
    pred_p = bag.predict(Xp)
    mae_p = regression_metrics(y_val, pred_p)['MAE']
    importances.append(mae_p - baseline_mae)
imp_df = pd.DataFrame({'feature': FEATURE_COLS, 'mae_increase':
    importances}).sort_values('mae_increase', ascending=False)
print("\nPermutation importance (bagged trees, increase in MAE when shuffled):\n", imp_df)
imp_df.to_csv('permutation_importance.csv', index=False)

# ---------------- Error analysis ----------------
err = y_val - pred_bag
dev_val_fe['abs_err'] = np.abs(err)
dev_val_fe['pred'] = pred_bag
print("\nWorst 10 predictions (bagged trees):")
worst = dev_val_fe.sort_values('abs_err', ascending=False).head(10)
print(worst[['load_id','pickup','delivery','distance','equipment','posted_rate','pred','abs_err']])

print("\nMAE by equipment:")
print(dev_val_fe.groupby('equipment')['abs_err'].mean())

dev_val_fe['distance_bin'] = pd.cut(dev_val_fe['distance'], bins=[0,300,700,1500,2500,4000])
print("\nMAE by distance bin:")
print(dev_val_fe.groupby('distance_bin', observed=True)['abs_err'].mean())

# ---------------- Plots: predicted vs actual + residuals ----------------
fig, axes = plt.subplots(1, 3, figsize=(15,4.5))
axes[0].scatter(y_val, pred_bag, s=4, alpha=0.2, color='#3B6FA0')
lims = [0, max(y_val.max(), pred_bag.max())]
axes[0].plot(lims, lims, 'r--', lw=1)
axes[0].set_xlabel('Actual posted_rate'); axes[0].set_ylabel('Predicted')
axes[0].set_title('Bagged Trees: Predicted vs Actual')

axes[1].scatter(pred_bag, err, s=4, alpha=0.2, color='#3B6FA0')
axes[1].axhline(0, color='r', ls='--', lw=1)
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('Residual (actual-pred)')
axes[1].set_title('Residuals vs Predicted')

model_names = list(results.keys())
mae_vals = [results[m]['MAE'] for m in model_names]
axes[2].barh(model_names, mae_vals, color='#3B6FA0')
axes[2].set_xlabel('MAE ($)'); axes[2].set_title('Model comparison (holdout MAE)')

plt.tight_layout()
plt.savefig('model_eval.png', dpi=130)
```

```
print("\nSaved model_eval.png")
```

03_final_predict.py

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from features_models import (fit_imputers, engineer_features, to_matrix,
                             BaggedTreesNumpy, RidgeRegressionNumpy, regression_metrics,
    FEATURE_COLS)

np.random.seed(42)
t0 = time.time()

train = pd.read_csv("train-test.csv", parse_dates=["date"])
val   = pd.read_csv("validation.csv", parse_dates=["date"])
tmpl  = pd.read_csv("validation-predictions-template.csv")
chart = pd.read_csv("december-chart-inputs.csv", parse_dates=["date"])

# ---------------- Fit imputers + winsorization bounds on FULL training data ----------------
stats = fit_imputers(train)
train_fe = engineer_features(train, stats)

lo, hi = train_fe['posted_rate'].quantile([0.005, 0.995])
y_full = np.clip(train_fe['posted_rate'].to_numpy(), lo, hi)
X_full = to_matrix(train_fe)

print(f"Training final model on all {len(train_fe)} rows (target winsorized to [{lo:.1f},
    {hi:.1f}])")

final_model = BaggedTreesNumpy(n_estimators=25, max_depth=8, min_samples_leaf=60,
                               min_samples_split=120, max_features=10, random_state=42)
final_model.fit(X_full, y_full)
print(f"Trained in {time.time()-t0:.1f}s")

# also refit linear model on full data, for the coefficient table in the report
lin_full = RidgeRegressionNumpy(alpha=5.0).fit(X_full, y_full)

# ---------------- Predict on validation.csv ----------------
val_fe = engineer_features(val, stats)
X_val = to_matrix(val_fe)
val_pred = final_model.predict(X_val)
val_pred = np.clip(val_pred, 1, None)  # rates can't be <= 0

out = tmpl.copy()
assert (out['load_id'] == val['load_id']).all()
out['predicted_rate'] = np.round(val_pred, 2)
out.to_csv('validation_predictions.csv', index=False)
print("\nSaved validation_predictions.csv:", out.shape)
print(out.head())
print("\npredicted_rate summary:\n", out['predicted_rate'].describe())

# sanity: compare predicted rate-per-mile distribution to training rate-per-mile
val_fe['pred'] = val_pred
val_fe['pred_rpm'] = val_fe['pred'] / val_fe['distance']
print("\nValidation predicted rate-per-mile summary (sanity check vs train ~2.0-2.4):\n",
      val_fe['pred_rpm'].describe())
print("\nValidation predicted rate-per-mile BY equipment:\n",
      val_fe.groupby('equipment')['pred_rpm'].mean())

# ---------------- December chart: build a city -> lat/lon lookup from all known cities
    ----------------
city_geo = pd.concat([
    train[['pickup','pickup_lat','pickup_lon']].rename(columns={'pickup':'city','pickup_lat':'lat','
    pickup_lon':'lon'}),
    train[['delivery','delivery_lat','delivery_lon']].rename(columns={'delivery':'city','delivery_la
    t':'lat','delivery_lon':'lon'}),
    val[['pickup','pickup_lat','pickup_lon']].rename(columns={'pickup':'city','pickup_lat':'lat','pi
    ckup_lon':'lon'}),
    val[['delivery','delivery_lat','delivery_lon']].rename(columns={'delivery':'city','delivery_lat'
    :'lat','delivery_lon':'lon'}),
]).drop_duplicates(subset='city').set_index('city')

missing_cities = set(chart['pickup']).union(chart['delivery']) - set(city_geo.index)
```

```
print("\nDecember-chart cities missing coordinate lookup:", missing_cities)

chart_fe = chart.copy()
chart_fe['pickup_lat'] = chart_fe['pickup'].map(city_geo['lat'])
chart_fe['pickup_lon'] = chart_fe['pickup'].map(city_geo['lon'])
chart_fe['delivery_lat'] = chart_fe['delivery'].map(city_geo['lat'])
chart_fe['delivery_lon'] = chart_fe['delivery'].map(city_geo['lon'])
# market_index / quote_signal are not provided for this scenario file at all,
# so we impute them at the training median (documented assumption -- see report)
chart_fe['market_index'] = stats['market_median']
# quote_signal has no fitted imputer (never missing in training) -> use training median directly
chart_fe['quote_signal'] = train['quote_signal'].median()

chart_full_fe = engineer_features(chart_fe, stats)
X_chart = to_matrix(chart_full_fe)
chart_pred = final_model.predict(X_chart)
chart_out = chart.copy()
chart_out['predicted_rate'] = np.round(chart_pred, 2)
chart_out.to_csv('december_chart_predictions.csv', index=False)
print("\nSaved december_chart_predictions.csv")
print(chart_out[['date','predicted_rate']])

fig, ax = plt.subplots(figsize=(9,4.5))
ax.plot(chart_out['date'], chart_out['predicted_rate'], marker='o', ms=3, color='#3B6FA0')
ax.set_title('Predicted rate — Lexington to Fort Wayne, Dry Van, 32,000 lb — December 2025')
ax.set_xlabel('Date'); ax.set_ylabel('Predicted rate ($)')
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('december_chart.png', dpi=130)
print("Saved december_chart.png")
print(f"\nTotal runtime: {time.time()-t0:.1f}s")
```

End of document. Deliverables referenced above: validation_predictions.csv, december_chart_predictions.csv, model_comparison.csv, linear_coefficients.csv, permutation_importance.csv, REPORT.md, README.md.